ForthLLM Development Report

Phases 4a & 4b • llama.vm / llama.cpp • 2026-10-31
Scope: committing the Phases 1–3 ForthLLM work, replacing the placeholder scaffold (4a), wiring `max_stack_depth`/`eviction` into the compaction (4b), and the DeepSeek-V4 (MLA) finding that re-scopes 4c. This report includes the reasoning behind each decision.

Contents

1. Executive summary • 2. Fork question • 3. Phase 4a — scaffold replacement • 4. DeepSeek-V4 finding (re-scopes 4c) • 5. Phase 4b — max_stack_depth + eviction • 6. End-to-end validation • 7. Commits & artifacts • 8. Remaining work (4c) • Appendix A — Decision log (the thinking)

1. Executive summary

This session closed out the ForthLLM KV-cache-compaction effort as follows:

- **Committed** the Phases 1–3 work (flags, engine, KV surgery, server hooks) that had been validated end-to-end but was uncommitted in both repos.
- **Answered the fork question**: fork llama.cpp (yes), merge the llama.vm pager into it (no).
- **Completed Phase 4a**: replaced llama.vm's token-level placeholder scaffold with a faithful port of the tested llama.cpp engine. Full test suite green (11/11).
- **Completed Phase 4b**: wired `--forthllm-max-stack-depth` and `--forthllm-eviction` into the compaction, validated end-to-end on a 27B model.
- **Re-scoped Phase 4c** after discovering that DeepSeek-V4's KV cache is a raw-SWA + compressed-block + ring-state structure, not a plain MLA cache — so 4c needs a design pass, not a mechanical port.

All work is committed, the feature patch is regenerated and in sync with the tree, and the docs (`AGENTS.md`, `forthllm-design.md`, the investigation doc) were updated to match.

2. Fork question

Question: "Since we don't own llama.cpp, should we fork it and just merge llama.vm into it?"

Recommendation: fork llama.cpp — yes; merge the llama.vm pager into it — no.

> Thinking ForthLLM is a *llama.cpp* feature (flags in `common/`, engine + KV surgery in the core, hooks in `tools/server`). Its durable home is a feature branch of llama.cpp, not a patch file that lives in another repo. A local feature branch *is* the minimal fork; a GitHub fork is only needed if we want to push/share, and `gh` is not authenticated in this environment, so the local branch is the fork for now — rebase-ability onto upstream is preserved either way.

> The pager, by contrast, is *deliberately* independent: it has its own build, is loaded at runtime via `dlopen` as `libggml-llamavm-{cpu,gpu}.so` into *any* llama.cpp build, and has a standalone test suite (`test_pager`, `test_gguf`, `llama-vm bench|tiered`). Merging it into llama.cpp would entangle two build systems and destroy the standalone use-case that is a core part of the project's objective. The integration point is already clean (`tensor_buft_overrides` + the dlopen shim). So: keep two repos, and let `llama.vm/patches/` record the llama.cpp-side work.

3. Phase 4a — replace the placeholder scaffold

What was done: replaced llama.vm's token-level placeholder (`forthllm::context_manager::process_chunk(tokens)`, which filled "summaries" with `sin(i + chunk_id) * 0.1f` and never touched a model) with a faithful port of the tested llama.cpp engine into `namespace forthllm`:

- `summarize()` — real KV-row pooling (attention / mean / last) into S summary rows.
- `stack` — chunk stack with lru / lfu / fixed-window eviction.
- `squeezer` — JL projection → per-vector min/max 8-bit quantization → BWT (the old `[-1,1]` quantization assumption is gone; values outside `[-1,1]` now survive).
- `bwt_transform` — `uint8_t` comparator + exact LF-mapping invert (the bytes≥128 case is now tested; the old invert was broken).
- `jl_projector` — deterministic Gaussian N(0, 1/output_dim) from a seed.

The token-level `context_manager`/`chunk_summary` API was removed (it had no real semantics); the KV-row API (`summarize` + `stack`) replaces it. `fm_index` was kept as a standalone utility.

`test_forthllm` (8 tests) and `test_squeezer` (6 tests) were rewritten to test real behavior.

> Thinking Two implementation options were on the table: **(a)** a thin re-export that calls llama.cpp's C API, or **(b)** porting the self-contained math into llama.vm. Option (a) was rejected because the real engine's API is KV-row-level and its C API needs a `llama_context*` — a re-export would make llama.vm's ForthLLM depend on a live llama.cpp context and break the repo's standalone value. Option (b) works because the engine's model-agnostic math depends only on `llama.h` (for `llama_pos`) and std headers — **no ggml** — so it ports cleanly and keeps llama.vm self-contained and unit-testable.

> The token-level `context_manager` could not be made "real" without a model (you cannot summarize raw token ids into an embedding without weights), so keeping it would have meant keeping a lie. Removing it and exposing the KV-row API is the honest port. One interaction bug was caught by the tests: the new BWT appends a 4-byte row index for invertibility, which corrupted `fm_index` (it consumed the BWT as its index) — a SIGSEGV and a wrong count. Fix: `fm_index` strips the 4-byte suffix. A residual, pre-existing quirk remains: `fm_index::locate` positions are off for a circular BWT without a sentinel; `count` is correct, and the test asserts count + position count, not exact positions.

Result: full ctest suite green (11/11). Committed as `1bbbb36`.

4. DeepSeek-V4 finding (re-scopes 4c)

The user's actual model (DeepSeek-V4-Flash, the `test-forth-deepseek()` target) uses `llama_kv_cache_dsv4`. Investigating it for 4c revealed it is **not** a standard MLA cache:

- `kv_raw` — a `llama_kv_cache_iswa` (normal raw/SWA token cache; the sliding-window half is what DSV4 raw attention reads).
- `kv_csa` / `kv_hca` / `kv_lid` — three **compressed K-only block caches** (storage only; visibility and block planning live in `llama_kv_cache_dsv4_context` / `llm_graph_input_dsv4`).
- `csa_state` / `hca_state` / `lid_state` — **ring compressor states** (row id = `pos % ratio`); the compressed rows are **graph outputs** (per-ubatch `comp_plan` recipes), not stored rows.

> Thinking Two facts make 4c a design problem rather than a port. First, the C API rejects DSV4 because `llama_forthllm_compact` does `dynamic_cast<llama_kv_cache*>(memory)` and `llama_kv_cache_dsv4` derives from `llama_memory_i`, not `llama_kv_cache` — so the cast fails and it logs "not supported". Second, `llama_kv_cache_dsv4::get_can_shift()` is hard-disabled ("compressed row metadata uses block-derived positions"), so the K-shift re-rotation machinery the Qwen3.5 path relies on is unavailable. Conceptually, DSV4

> *already* compresses its KV (the block caches are the compression); ForthLLM on top would be compression-on-compression, and the ring states are graph-computed, not rows we can summarize in place. Realistic targets, in decreasing confidence: (i) compact the `kv_raw` SWA cache (bounded window — small win), (ii) operate on the compressed block caches (requires re-deriving ring states — graph work), (iii) a DSV4-specific compaction that respects the `comp_plan`/`ratio` structure. Conclusion: 4c needs its own design pass before implementation; do *not* attempt it as "just make the cast succeed".

5. Phase 4b — wire max_stack_depth + eviction

What was done: the `--forthllm-max-stack-depth` and `--forthllm-eviction` flags now take effect.

- `llama_forthllm_params` gains `max_stack_depth` + `eviction`, carried from `common_params_forthllm` through `common_forthllm_compact`. `max_stack_depth <= 0` keeps the previous no-limit behavior.
- `llama_kv_cache::forthllm_compact` computes `n_keep = min(n_chunks, max_stack_depth)` and `n_drop = n_chunks - n_keep`. In the oldest→newest chunk loop, the oldest `n_drop` chunks are **dropped entirely** (all C cells freed, subsequent cells shifted left by C, no summary rows); the most recent `n_keep` chunks are summarized as before (C−S freed).
- A dropped chunk is reported as `{pos0, n_tokens=C, n_freed=C}` (so `n_summary = 0`); the server token-sync now counts filled entries (not `n_freed/(C−S)`) and erases `[pos0 + (n_tokens − n_freed), pos0 + n_tokens)` per chunk — the whole chunk when dropped, `[pos0+S, pos0+C)` when summarized.

> Thinking The original plan was to drive eviction from a per-slot `llama_forthllm_stack` in the server hook. That was rejected on inspection: the compaction *re-derives* the layout from the cells on every call (it has no persistent notion of which cells are summaries), so a persistent stack in the server would not match what the cache actually does. Integrating the eviction *into* the compaction — dropping the oldest chunks during the same oldest→newest pass that summarizes the rest — is simpler and correct, and reuses the exact validated primitives (`seq_rm`, `pos_add`, head tracking) with only the freed/shifted count changed (C instead of C−S). Cell indices are stable across the shifts (only positions move, and they all move together, so contiguity is preserved), which is what makes the drop branch safe.

> Two consequences had to be handled downstream. The chunk count can no longer be derived as `n_freed/(C−S)` (dropped chunks free C, not C−S), so the server counts filled

> `chunk_info` entries instead. And the per-chunk erase range must use `keep = n_tokens − n_freed` (which is S for summarized chunks and 0 for dropped ones) rather than a global S.

> The `squeezer`/`jl_dim` were deliberately *not* plumbed into the C API: they describe the stored/offloaded representation (in-cache summary rows stay head-dim), so their consumer is state save/load, which does not exist yet. Adding unused fields to a public C struct would be sloppy; they remain carried in `common_params_forthllm` and are documented as future. All three eviction policies currently reduce to "drop the oldest" — importance-based (lfu) eviction needs per-chunk attention counts, which is a callback into the attention path and is deferred.

Result: E2E-validated (below). Committed as `342c7c4f3`.

6. End-to-end validation

Phase	Validation	Result
4a	`test_forthllm` (8 tests: summarize last/mean/attention, S-clamp, lru + fixed-window eviction, parse helpers, config validation) and `test_squeezer` (6 tests: JL determinism, BWT ASCII + high-byte round-trip, FM-index count, squeezer size, min/max scaling)	**All pass; full ctest 11/11**
4b	Qwen3.8-27B UD-Q4_K_M (qwen35 / hybrid / IMROPE), CPU, 4096 ctx, `--forthllm-max-stack-depth 4`, a ~2196-token prompt ending in a self-contained factual question	**Compacted 16 chunks = 12 dropped + 4 summarized = 2040 cells freed, 2196 → 156 tokens; answered coherently ("Paris")**

The 4b numbers check out exactly: 12×128 (dropped) + 4×126 (summarized, C−S = 128−2) = 1536 + 504 = **2040**, and 2196 − 2040 = **156** remaining cells. Because the answer stayed coherent after dropping the first 1536 tokens, the drop-branch position bookkeeping (free + shift + K-shift re-rotation) is confirmed correct — the question lives in the kept raw window.

```
server log:  forthllm: compacted 16 chunk(s), freed 2040 cells, prompt 2196 -> 156 tokens
```

```
answer:      "Paris" (coherent)
```

7. Commits & artifacts

Repo	Branch	Commit	Subject
llama.cpp	`forthllm`	`26833e236`	ForthLLM: KV-cache compaction engine + server hooks (Phases 1–3)
llama.cpp	`forthllm`	`342c7c4f3`	ForthLLM: wire max_stack_depth + eviction into the compaction (Phase 4b)
llama.vm	`main`	`f0c1707`	ForthLLM: record llama.cpp implementation + validation
llama.vm	`main`	`1bbbb36`	ForthLLM: replace placeholder scaffold with the real engine (Phase 4a)
llama.vm	`main`	`39b730b`	ForthLLM: record Phase 4b (max_stack_depth + eviction wired)

- `llama.cpp` `master` is untouched at upstream `36a0e2ffc`; the stray untracked `asdf` (an old type-66 stub diff) was deliberately left out of every commit.
- Feature patch `patches/llama-cpp-forthllm-kv-compaction.patch` regenerated to include 4b: **12 files, 1441 insertions** (was 1393); `git apply --reverse --check` passes against the tree (in sync).
- Docs updated in both repos: `AGENTS.md`, `forthllm-design.md`, and the investigation doc's Phase 4 handoff.

Note: git identity was auto-set to `alfred@cuda-mx`; set `user.name`/`user.email` and `git commit --amend --reset-author` if a different author is wanted.

8. Remaining work — Phase 4c (MLA / DeepSeek-V4)

4c is the user's actual model and the highest-value item, but it is the largest and needs a design pass first. Three questions to settle before implementation:

1. **Which cache to compact?** The raw SWA cache (bounded window — small win) vs.

the compressed block caches (already compressed — ForthLLM would be a second compression).

2. **How do ring states re-derive after compaction?** The `csa/hca/lid` states are graph-computed ring states (`pos % ratio`); dropping/summarizing rows must keep them consistent, which is graph work, not row surgery.
3. **Is K-shift re-rotation available at all?** `get_can_shift()` is currently hard-disabled for DSV4; the Qwen3.5 path's re-rotation of shifted keys has no DSV4 equivalent yet.

Appendix A — Decision log (the thinking)

#	Decision	Reasoning / rejected alternative
1	Fork llama.cpp = yes	ForthLLM is a llama.cpp feature; a feature branch is its durable home. Local branch is the fork now (no `gh` auth); GitHub fork is a later step.
2	Merge the pager into llama.cpp = no	The pager is intentionally independent (own build, dlopen'd, standalone tests). Merging entangles builds and kills standalone use. Keep two repos; `patches/` records the llama.cpp side.
3	4a: port the math, don't re-export	A re-export needs a `llama_context*` and breaks standalone-ness. The engine math is ggml-free, so it ports cleanly into `namespace forthllm`.
4	4a: remove `context_manager`	The token-level placeholder has no real semantics (can't summarize token ids without a model); keeping it would keep a lie. Expose the KV-row API instead.
5	4a: keep `fm_index`, strip BWT suffix	`fm_index` is a tested standalone utility. The new BWT's 4-byte row index corrupted it (SIGSEGV); stripping the suffix fixes `count`. `locate` positions remain a pre-existing circular-BWT quirk (documented, not chased).
6	4b: integrate eviction into the compaction	A per-slot stack in the server doesn't match the cache (the compaction re-derives layout each call). Dropping the oldest chunks in the same pass reuses validated primitives and is correct.
7	4b: report dropped chunks as `{pos0, C,`	Reuses the existing server erase logic via `keep = n_tokens − n_freed` (0 when dropped). Avoids a new C-API out-param for the

	`C}`	chunk count.
8	4b: don't plumb squeezer/jl_dim yet	They describe the stored/offloaded representation; the consumer (state save/load) doesn't exist. Adding unused fields to a public C struct is sloppy — carry them in `common_params_forthllm`, document as future.
9	4c: design pass before code	DSV4 is raw-SWA + compressed-block + ring-state, not plain MLA; the C API cast fails and K-shift is disabled. "Just make the cast succeed" would be wrong — settle which cache to compact, ring-state re-derivation, and K-shift availability first.

Generated 2026-10-31 from the llama.vm / llama.cpp ForthLLM development session. Source of record: `llama.vm/llama-server-forthllm-investigation.md` (Phase 4 handoff) and the commits listed in §7.